Requirements and prototyping

Lecture 2, IT Product Development

Innopolis University, 2026

Agenda

- Why this week
- The chain: vision, stories, criteria, prototype
- Prototyping
- Validating with the customer

Why this week

Last week

You settled:

- Which products exist, and how well each one does the job
- Where they all fall short: `GAP-01`, `GAP-02`
- What you will do differently: `VP-01`, `VP-02`
- What the customer disagrees with

You did not settle:

- What your product actually has to do
- What it will not do
- What you are building first

This week: say what it must do

The research said **what to build**.

This week you say **what it must do**, and you find out how much of that was wrong.

Conjectures lead to experiments. Experiments lead to facts. Facts lead to decisions.

Quiz

- What did last week’s research give you that you can build on?
- Name one thing it did not give you.

The chain

The chain

GAP-nn, VP-nn -> the vision -> **US-nn** -> the prototype -> the customer -> **a change**

Every link answers one question:

- The vision: what is this for?
- A user story: what does one user need, for which `VP-nn` of the goal?
- Acceptance criteria: how would we know?
- The prototype: are we guessing?
- The meeting: where are we wrong?

The product vision

The vision is the one page a reader holds to understand what you decided to build. It has five parts:

- The **goal**: what the product must achieve
- The **constraints**: what you cannot change, and what each one costs
- The **stakeholders**: who uses it, who operates it, who is affected
- The **boundary**: what it will not do, and who does it instead
- A **system context diagram**: your system in the environment

The goal traces to a `VP-nn`. A goal that supports no value proposition is a feature you would like.

Constraints and assumptions

Constraint: a condition you cannot change.

Assumption: a belief you have not verified but treat as true for now; you can be wrong about it.

Record every constraint under one of four types:

- **Customer-given**: e.g. integration with a specific API, limited budget
- **Team-given**: e.g. the team size and skills
- **Environmental**: e.g. the weeks left in the course
- **Derived**: what follows from the other constraints

If you don’t know what a constraint costs you, you don’t understand it.

The boundary

Boundary: the line between what your product does and what the people and systems around it do.

The context diagram **draws** the line. The vision **writes down** the parts of it someone could argue with: one item per job the product will not do, who does it instead, and why. Each item has its own identifier, so a story or a decision can cite it.

```
BND-01
Host the video call.

- Status: Active
- Handled by: Video service
- Why: CON-03, the single-term course:
  video stays an integration.
```

```
BND-02
Schedule more than one expert at a time.

- Status: Active
- Handled by: Nobody
- Why: DEC-002, the customer's decision:
  the experts work alone.
```

The context diagram

Context diagram: is a picture of the product at its edges:

- The **product** itself
- The **external actors** that use it or are affected by it
- The **external systems** it exchanges data with

It stops there. No containers, no components, and no internal structure; the container view arrives in Week 4.

The diagram and the boundary must agree:

- Whatever the boundary says handles a job is on the diagram.
- Nothing on the diagram does a job the boundary leaves to nobody.

A user story

User story: captures a need of an actor together with the value.

```
As a <user>
I want to <action>
so that <value>
```

A user is any actor with a goal: the person the product serves, and the operator or administrator who keeps it running.

It is a **need**, not a design.

- “Avoid double-booking a slot” is a need.
- “Add a calendar page” is a feature you have already built in your head.

A story states the problem precisely and leaves the solution open.

If you cannot finish a user story with a value, you have written a task.

A non-story and a story

Both are about a meeting booking app. Both are made up.

Not a story:

> As an expert, I want a booking page with a calendar, so that my clients can see my available time.

It names the design (“a booking page with a calendar”) and stops at what the screen shows. Why does the expert need clients to see the free time? The value is missing.

A story:

> As a coach who sells sessions online, I want a client to pay when they book, so that an unpaid slot does not block a paying one for the rest of the week.

A more precise <user>, the value they get, and no design: a payment page, a deposit, or an invoice would all satisfy it.

How specific?

Ask **who settled the detail**.

- Only the team would decide it (a screen, a component, a library): leave it out.
- The user or the customer settled it: the story may carry it.

> As a coach, I want booked sessions to appear in the calendar I already use, so that I do not double-book a slot.

The customer says the coach uses Google Calendar, so an **acceptance criterion** names it. The need stays the same if the coach later switches calendars.

Google Calendar is on the context diagram, outside the product: a fact about the problem, not your design. Stories get more specific as they get closer to being built.

Acceptance criteria

The story says **why**. The criteria say **what success looks like** (when the story is completed).

Two or more per story, and the test for a good acceptance criterion is simple: **could somebody who is not you run the check and get the same answer?**

A criterion may name a screen, a field, or a system state, because that is what an observer sees. The story may not, because that would be the design.

“Works well” is not a criterion. A criterion you cannot run is a belief.

A criterion that is hard to write is not a writing problem. It means you do not yet know what the product does in that case, which makes it a good thing to prototype.

Prioritizing with MoSCoW

Priority is a four-way scale, relative to the product you intend to finish:

- **Must Have**: the product is not the product without it
- **Should Have**: important, but the product is still coherent without it
- **Could Have**: valuable, and the first thing to cut if you can’t implement for some reason
- **Won’t Have**: a real need you have deliberately excluded

Each label should be justified and account for constraints.

A `Won't Have` reason may cite the boundary; nothing you build contradicts it.

Quiz

- What question does each artifact in the chain answer?

  GAP-nn, VP-nn -> the vision -> **US-nn**
- What does the chain give you that a list of features does not?
- What is the difference between a need and a design?
- How do you tell a real acceptance criterion from a wish?

Prototyping

Proof of concept (PoC), prototype, MUP, MVP

Proof-of-concept (PoC): “Can this work at all technically?”

Prototype: “How will this look? Does this user flow make sense?”

Minimum usable product: “Can the user complete the core tasks without getting frustrated?”

Minimum viable product: “Will people use it?” The minimum ready-to-ship product that can deliver value to the users.

The prototype

A prototype is an instrument for finding out what is wrong and killing wrong ideas early.

It is not a showcase, and it is not a part of the product.

Ask one question first: **which user stories am I least sure about?**

Then build the cheapest thing that gets a reaction to that question from relevant stakeholders (customer - in case of ITPD).

After you have learned with the prototype what you planned, you can throw it away.

Forms of prototype

Match the fidelity to the question, not to your ambition.

- **Paper or static image**: fastest, for layout, vocabulary, and “is this the right problem”
- **Clickable design**: for a flow you want the customer to move through
- **Code spike**: for a technical risk, kept off `main` and thrown away after

A high-fidelity prototype is the wrong tool for “does the customer recognise this problem”: you will spend two days on it and they will comment on the colour.

Quiz

- What question does the prototype answer?
- What is the difference between a prototype and an MUP?

Validating with the customer

What the second meeting asks

The target is one sentence: **which of these is wrong?**

The prototype, the boundary, and the candidate for the MUP.

Show the candidate next to every user story and its priority, so the customer can move a story into or out of it.

Walk through each story’s acceptance criteria if time allows.

Do not treat agreement as a result

“That sounds great” about your own idea has told you almost nothing.

If the customer agrees with everything, you showed them the answer rather than the question.

Therefore, the `Disagreements` table in your meeting report should not be empty.

If there are no disagreements, you probably showed what you were already sure about.

Quiz

- What is the target of the meeting, in one sentence?
- What do you do when the customer approves everything?

Questions?